ACADEMIC PRACTICAL REPORT

ADVANCED WEB TECHNOLOGIES (AWT)

PRACTICAL 9

Designing Test Suites and Estimating Test Coverage Metrics with Structural Complexity

Application Case Study:

Perfect Rent & Remind
Smart Clothing Rental Management System

Layer / Component	Implemented Technologies & Tools
Backend Framework	Node.js (v25.9.0), Express.js (v5.3.0), Morgan, Helmet, CORS
Database & ODM	MongoDB (Replica Set Supported), Mongoose ODM (v8.9.5)
Testing Harness & SQA	Jest (v30.5.2), Supertest (v7.3.1), MongoMemoryReplSet (v11.3.0)
Security & Tokens	JWT (jsonwebtoken v9.0.3), bcryptjs (v3.0.3), express-rate-limit
Frontend Client	React 19, TailwindCSS v4, Vite v5.4, Lucide Icons, React Router 7

Laboratory Report • Academic Term: 2026–2027 • Date: October 2026

Aim of the Practical

To systematically design, organize, and execute automated test suites for a full-stack web application (Perfect Rent & Remind: Smart Clothing Rental Management System), analyze structural program complexity using McCabe's Cyclomatic Complexity metric, establish structured test scenarios, formulate positive and negative test cases covering boundary and exceptional conditions, and empirically measure and interpret code coverage metrics (statement, branch, function, and line coverage) using Jest and Istanbul.

1. Principles of Test Suite Design

A test suite is a formalized, hierarchically structured collection of test cases designed to validate that a software system complies with specified functional, non-functional, and security requirements. In modern web application engineering, test suite design adheres to the 'Testing Pyramid' paradigm, striking an optimal balance between execution velocity, diagnostic precision, and end-to-end operational fidelity.

By decomposing testing activities across granular tiers—from isolated domain functions up to multi-service integration—software engineers prevent defect escapes, preserve regression safety, and ensure transactional integrity under concurrent user workflows.

Testing Level	Scope & Architectural Purpose	Related Project Modules	Implementation Status & Test Files
Unit Testing	Validates isolated domain algorithms, mathematical utilities, and individual validation rules without external I/O, database connections, or HTTP mocking.	calculateRental.js calculateFine.js generateToken.js Joi Validators	**IMPLEMENTED & VERIFIED (tests/middleware.test.js, inline validator tests)**
Integration Testing	Verifies interaction between controllers, service layers, Mongoose schemas, and database transactions using in-memory MongoDB replica sets.	booking.service.js payment.service.js return.controller.js auth.middleware.js	**IMPLEMENTED & VERIFIED (tests/booking.test.js, payment.test.js, return.test.js)**
API / System Testing	Validates complete HTTP endpoints, status codes, payload formatting, authorization headers, and error envelopes over Supertest agent.	All Express API Routers (/api/auth, /api/clothing, /api/bookings, /api/payments)	**IMPLEMENTED & VERIFIED (tests/auth.test.js, clothing.test.js, category.test.js)**
Security & Non-	Ensures role-based	auth.middleware.js	**IMPLEMENTED & VERIFIED**

Functional	access control (RBAC), prevents role tampering, evaluates rate limiting, and assesses database transaction rollback safety.	rateLimiter.middleware.js transactionHelper.js	**(tests/e2e.integration.test.js, rateLimiter warnings documented)**

[ISOLATION ARCHITECTURE] *All test files in Perfect Rent & Remind execute in an isolated runtime powered by MongoMemoryReplSet. This completely eliminates external database dependencies, prevents test data contamination, and allows full multi-document transaction testing.*

2. Structural Complexity and Test Coverage Metrics

2.1 McCabe's Cyclomatic Complexity

Cyclomatic complexity, formulated by Thomas J. McCabe (1976), is an algorithmic metric that measures the structural complexity of software code by quantifying the number of linearly independent paths through its Control Flow Graph (CFG).

Mathematical Formulas:
1. For general programs with P connected components: V(G) = E - N + 2P
2. For a single connected control-flow graph (P = 1): V(G) = E - N + 2
3. Predicate Node Counting Equivalence: V(G) = P_pred + 1
   (where P_pred is the count of binary decision points: if, while, for, &&, ||, ternary ? :)

Function / Method	Source File	Decision Points (P_pred)	V(G)	Risk Level	Testing Implications (Basis Paths)
normalizeUtcDate	src/utils/calculateRental.js	if (string), if (parts.length == 3)	**3**	Low (1-10)	3 basis paths: YYYY-MM-DD string, non-split string, Date object.
calculateRentalDuration	src/utils/calculateRental.js	0 (pure linear arithmetic)	**1**	Low (1-10)	1 path: linear formula (returnDate - startDate)/86400000.
calculateFine	src/utils/calculateFine.js	0 (uses Math.max without explicit if)	**1**	Low (1-10)	1 path: UTC difference calculation and linear fee multiplication.

protect (auth)	src/middleware/auth.middleware.js	headers && startsWith, !token, !req.user, catch	**5**	Low (1-10)	5 basis paths: valid Bearer, missing header, bad token, deleted account, expired.
admin (guard)	src/middleware/auth.middleware.js	req.user && req.user.role === 'ADMIN'	**3**	Low (1-10)	3 basis paths: admin role, customer role, unauthenticated caller.
calculateReservedQuantity	src/services/booking.service.js	ternary session ? session : {}	**2**	Low (1-10)	2 basis paths: query with transaction session vs. standalone query.
createBooking	src/services/booking.service.js	13 predicates (dates, stock, retry loop, conflict)	**16**	Moderate (11-20)	16 basis paths required. High branching due to concurrency retries & validation.
cancelBooking	src/services/booking.service.js	!booking, role!=ADMIN && custId!=user, !cancellable	**5**	Low (1-10)	5 basis paths: booking 404, owner cancel, admin cancel, forbidden, terminal status.
updateBookingStatus	src/controllers/booking.controller.js	18 predicates (state transitions, payment checks)	**19**	Moderate (11-20)	19 basis paths required. Complex state machine (PENDING->CONFIRMED->PICKED_UP).
createPayment	src/services/payment.service.js	17 predicates (simulated fails, COP, UPI, catch 11000)	**18**	Moderate (11-20)	18 basis paths required. Handles electronic vs. cash flows & duplicate keys.
refundPayment	src/services/payment.service.js	!payment, isRefunded, notPaid, !cancelled, reason	**7**	Low (1-10)	7 basis paths: 404, double refund, uncollected cash, cancelled booking, reason tag.

processReturn	src/controllers/return.controller.js	!booking, duplicate, !picked_up, pastDate, condition, late	**10**	Low (1-10)	10 basis paths: intake validation, late fee branch, condition classification.
settleReturn	src/controllers/return.controller.js	!record, settled, damage > deposit, supportsTx	**7**	Low (1-10)	7 basis paths: idempotency (409), deposit damage deduction, atomic completion.

2.2 Test Coverage Metrics

Measured Test Coverage Results:
The following empirical coverage values were recorded directly from the Jest/Istanbul coverage reporter following test execution across 10 test suites containing 86 automated test cases on October 23, 2026:

Subsystem / Directory	Statement Coverage (%)	Branch Coverage (%)	Function Coverage (%)	Line Coverage (%)
ALL REPOSITORY FILES	**76.88%**	**59.15%**	**85.22%**	**77.25%**
src/ (Application Root)	88.23%	50.00%	0.00%	88.23%
src/config/ (Environment)	100.00%	70.83%	100.00%	100.00%
src/controllers/ (API Controllers)	74.58%	55.50%	87.50%	75.41%
src/middleware/ (Security & Guards)	79.03%	69.38%	77.77%	80.00%
src/models/ (Mongoose Schemas)	91.42%	85.71%	100.00%	91.42%
src/routes/ (Express Route	95.06%	11.11%	0.00%	95.06%

Mappers)				
src/services/ (Booking & Payment Logic)	64.33%	60.00%	90.47%	64.21%
src/utils/ (Calculators & Helpers)	98.11%	69.23%	100.00%	98.11%
src/validators/ (Joi Schemas)	100.00%	50.00%	100.00%	100.00%

[COVERAGE & 501 GUARD INTERPRETATION] *Coverage Interpretation & 501 Guard Distinction: Overall statement coverage reaches 76.88% and function coverage reaches 85.22%. Branch coverage sits at 59.15% primarily due to defensive error paths: transaction retry exhaustion in booking.service.js, circular dependency fallbacks, and Phase 1 HTTP 501 defensive guards. Tests passing on HTTP 501 verify that the API safely rejects unsupported operations; they do NOT imply that the deferred features are implemented.*

3. Test Scenarios

Based on the implemented features, business rules, and API endpoints of Perfect Rent & Remind, exactly 12 master test scenarios (TS-01 to TS-12) have been formulated. Each scenario encapsulates an end-to-end user journey or core operational requirement.

Scenario ID	Module / Domain	Scenario Title	Testing Objective & Scope
TS-01	Authentication	**Customer Registration & Token Issuance**	Verify new customer registration, password hashing, role enforcement, login, and JWT issuance.
TS-02	Authorization	**Role-Based Access Control (RBAC)**	Validate token extraction, rejection of invalid/expired tokens, and customer vs. admin route guarding.
TS-03	Clothing Catalog	**Clothing Browsing, Filtering & Constraints**	Ensure public catalog discovery and verify that deferred operations (date queries, deletion, reduction) safely return

			HTTP 501.
TS-04	Booking Engine	**Inventory Reservation & Date Validation**	Validate booking creation with dynamic future dates, past-date rejection, and atomic concurrency overlap prevention.
TS-05	Pricing Engine	**Rental Cost & Security Deposit Computation**	Verify mathematical computation of rental duration, linear charges, flat deposits, and total price.
TS-06	Booking Lifecycle	**Customer Booking History & Cancellation**	Ensure booking ownership isolation, voluntary cancellation from PENDING/CONFIRMED, and stock release.
TS-07	Payments Engine	**Electronic Payment Processing (UPI/CARD)**	Verify checkout payment handling, generation of PAID payment records, and atomic booking confirmation.
TS-08	Cash on Pickup	**Cash-on-Pickup Initiation & Collection**	Validate PENDING payment creation without confirming booking, manual admin confirmation, and handover collection.
TS-09	Data Integrity	**Duplicate Payment Prevention & Rollbacks**	Enforce single payment per booking via unique index, race condition handling, and session rollback safety.
TS-10	Refunds	**Admin Refund Processing & Eligibility**	Validate admin-only refunds on PAID payments for CANCELLED bookings, and rejection of uncollected cash refunds.
TS-11	Physical Returns	**Return Logging, Fines &**	Verify admin return intake, calendar-day fine

		Data Masking	calculation (Rs 100/day), status assignment, and customer data masking.
TS-12	Settlement	**Security Deposit Settlement & Completion**	Verify admin damage charge deductions, deposit refunds, marking settled=true, and idempotent 409 rejections.

4. Positive and Negative Test Cases

To thoroughly evaluate the robust operation of Perfect Rent & Remind, detailed positive (P) and negative (N) test cases have been developed for all 12 operational scenarios. Every test case defines specific preconditions, execution steps, expected outcomes, actual execution results, and empirical pass/fail statuses reflecting real backend execution.

Scenario TS-01: Customer Registration and Authentication

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-01-P01	Positive	Customer registration with valid credentials	Email not registered	POST /api/auth/register with valid customer payload	HTTP 201 Created; role=CUSTOMER; token issued	HTTP 201 Created; token returned; password omitted	**Pass**
TC-01-P02	Positive	Customer login with valid credentials	Account exists	POST /api/auth/login with valid email and password	HTTP 200 OK; valid JWT token returned	HTTP 200 OK; token contains customer role claim	**Pass**
TC-01-N01	Negative	Privilege escalation prevention on register	None	POST /api/auth/register with payload role='ADMIN'	HTTP 201 Created; role forced to 'CUSTOMER'	HTTP 201 Created; backend forced role='CUSTOMER'	**Pass**
TC-01-N02	Negative	Reject duplicate email registration	Email exists	POST /api/auth/register with existing email	HTTP 400 Bad Request; code=AUTH_EMAIL_EXISTS	HTTP 400 Bad Request; AUTH_EMAIL_EXISTS returned	**Pass**

TC-01-N03	Negative	Reject login with invalid password	Account exists	POST /api/auth/login with wrong password	HTTP 401 Unauthorized; code=AUTH_INVALID_CREDENTIALS	HTTP 401 Unauthorized; AUTH_INVALID_CREDENTIALS	**Pass**

Scenario TS-02: Role-Based Access Control and Privilege Enforcement

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-02-P01	Positive	Admin access permitted on admin route	Admin token	POST /api/categories with admin Authorization header	HTTP 201 Created; category document persisted	HTTP 201 Created; category successfully created	**Pass**
TC-02-N01	Negative	Reject protected route when token is missing	None	GET /api/users/profile with no Authorization header	HTTP 401 Unauthorized; code=AUTH_TOKEN_MISSING	HTTP 401 Unauthorized; AUTH_TOKEN_MISSING	**Pass**
TC-02-N02	Negative	Reject protected route with invalid JWT token	None	GET /api/users/profile with header 'Bearer invalid.token'	HTTP 401 Unauthorized; code=AUTH_TOKEN_INVALID	HTTP 401 Unauthorized; AUTH_TOKEN_INVALID	**Pass**
TC-02-N03	Negative	Forbid customer from accessing admin route	Customer token	POST /api/categories with customer Authorization header	HTTP 403 Forbidden; code=FORBIDDEN	HTTP 403 Forbidden; message='Not authorized as an admin'	**Pass**

Scenario TS-03: Clothing Catalogue Browsing, Filtering, and Constraints

[HTTP 501 DEFERRED FEATURE DISTINCTION] *IMPORTANT ARCHITECTURAL DISTINCTION: For test cases TC-03-N01 through TC-03-N04, a status of 'Pass' verifies that the API accurately returns HTTP 501 NOT_IMPLEMENTED as a defensive contract guard. This confirms that the server safely rejects unsupported operations without corrupting database state. It does NOT indicate that the requested features (date-range filtering, clothing deletion, or quantity reduction) are functional. These features remain intentionally UNIMPLEMENTED / DEFERRED in Phase 1.*

Test	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status

ID							
TC-03-P01	Positive	Retrieve clothing with default pagination	Items in DB	GET /api/clothing	HTTP 200 OK; data.items array and pagination metadata	HTTP 200 OK; items and pagination returned. Feature fully implemented.	**Pass**
TC-03-P02	Positive	Filter clothing by size and price range	Items in DB	GET /api/clothing?size=M&minPrice=500&maxPrice=2000	HTTP 200 OK; all items match size=M and price bounds	HTTP 200 OK; query filters applied successfully. Feature fully implemented.	**Pass**
TC-03-P03	Positive	Retrieve single clothing item by valid ObjectId	Clothing record exists	GET /api/clothing/:id	HTTP 200 OK; data._id matches requested ID	HTTP 200 OK; returns full item entity. Feature fully implemented.	**Pass**
TC-03-N01	Negative	Deferred Feature Verification: Date availability query rejection	None	GET /api/clothing?startDate=...&endDate=...	API Contract: Returns HTTP 501 Not Implemented (code: NOT_IMPLEMENTED). Feature Status: Deferred in Phase 1.	API 501 Guard Verified: Returns HTTP 501. Feature Implementation Status: UNIMPLEMENTED (Passing test proves guard, not feature).	**Pass (501 Guard Verified; Feature Deferred)**
TC-03-N02	Negative	Deferred Feature Verification: Standalone availability route rejection	Item exists	GET /api/clothing/:id/availability	API Contract: Returns HTTP 501 Not Implemented. Feature Status: Deferred in Phase 1.	API 501 Guard Verified: Returns HTTP 501. Feature Implementation Status: UNIMPLEMENTED (Route safely blocked; feature not built).	**Pass (501 Guard Verified; Feature Deferred)**
TC-03-	Negative	Deferred Feature Verificati	Admin token	DELETE /api/clothing/:id with admin token	API Contract: Returns HTTP 501 Not	API 501 Guard Verified:	**Pass (501 Guard**

N03		on: Clothing deletion rejection			Implemented. Feature Status: Deferred per BR-040.	Returns HTTP 501. Feature Implementation Status: UNIMPLEMENTED (Deletion blocked to protect booking integrity).	**Verified; Feature Deferred)**
TC-03-N04	Negative	Deferred Feature Verification: Garment stock reduction rejection	Item qty=5	PUT /api/clothing/:id with lower quantity (qty=2)	API Contract: Returns HTTP 501 Not Implemented. Feature Status: Deferred in Phase 1.	API 501 Guard Verified: Returns HTTP 501. Feature Implementation Status: UNIMPLEMENTED (Reduction blocked to avoid overselling).	**Pass (501 Guard Verified; Feature Deferred)**

Scenario TS-04: Inventory Reservation, Rental Dates, and Overlap Prevention

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-04-P01	Positive	Create PENDING booking with valid future dates	Stock qty=5	POST /api/bookings with dynamic future dates (day +7 to +9)	HTTP 201 Created; status=PENDING; duration=2	HTTP 201 Created. Dynamic future dates verified in booking.test.js:82.	**Pass**
TC-04-P02	Positive	Allow adjacent non-overlapping bookings (half-open interval)	Booking A active	POST /api/bookings with adjacent interval (day +5 to +7 & +7 to +10)	HTTP 201 Created; half-open interval allows reservation without conflict	HTTP 201 Created. Dynamic dates verified in booking.test.js:179; booking on return day succeeds.	**Pass**
TC-04-	Positive	Handle concurrent booking	Stock qty=3	2 simultaneous	Exactly 1 request succeeds (201); 2nd rejected with	Tested in booking.test.js:206; statuses [201, 409];	**Pass**

P03		requests safely without overselling		requests each requesting 2 units for overlapping dates	HTTP 409	exactly 2 units reserved.	
TC-04-N01	Negative	Reject invalid rental dates where start >= return date	None	POST /api/bookings with start (day +9) and return (day +7)	HTTP 400 Bad Request; code=BOOKING_DATE_INVALID	HTTP 400 Bad Request; BOOKING_DATE_INVALID returned per BR-005.	**Pass**
TC-04-N02	Negative	Reject past rental start dates relative to current UTC date	None	POST /api/bookings with start=2020-01-01, return=2020-01-05	HTTP 400 Bad Request; code=BOOKING_DATE_INVALID	HTTP 400 Bad Request; BOOKING_DATE_INVALID returned. Verified in booking.test.js:122.	**Pass**
TC-04-N03	Negative	Reject requested quantity exceeding total physical stock	Stock qty=3	POST /api/bookings with dynamic future dates, requested qty=10	HTTP 400 Bad Request; code=BOOKING_QUANTITY_INVALID	HTTP 400 Bad Request; BOOKING_QUANTITY_INVALID. Dynamic dates reach quantity check.	**Pass**
TC-04-N04	Negative	Return 409 BOOKING_OVERLAP when stock is exceeded	Stock qty=3	POST /api/bookings requesting overlapping dates exceeding stock	HTTP 409 Conflict; code=BOOKING_OVERLAP	HTTP 409 Conflict; BOOKING_OVERLAP returned. Dynamic dates ensure clean conflict trigger.	**Pass**

Scenario TS-05: Rental Pricing Calculation and Financial Computation

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-05-P01	Positive	Exact pricing calculation for multi-day rental	Price=1000, Dep=2500	Create booking: 3 days, quantity=2	rentalAmount=6000, securityDeposit=2500, total=8500	Exact match: rentalAmount=6000, totalAmount=8500	**Pass**

TC-05-P02	Positive	Flat security deposit rule (no quantity multiplier)	Price=500, Dep=1500	Create booking: 1 day, quantity=4	rentalAmount=2000, securityDeposit=1500, total=3500	Verified: security deposit remains strictly flat (1500)	**Pass**
TC-05-N01	Negative	Reject rental duration of zero days	None	POST /api/bookings with rentalStartDate === returnDate	HTTP 400 Bad Request; code=BOOKING_DATE_INVALID	HTTP 400 Bad Request; caught by normStart >= normEnd	**Pass**

Scenario TS-06: Customer Booking History and Cancellation Workflow

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-06-P01	Positive	Customer retrieves only their own bookings	Cust A & B bookings	GET /api/bookings with Customer A token	HTTP 200 OK; array contains only Customer A records	HTTP 200 OK; strict isolation by customerId: req.user._id	**Pass**
TC-06-P02	Positive	Cancel PENDING booking and release inventory	Booking PENDING	PUT /api/bookings/:id/cancel, followed by re-booking	HTTP 200 OK; status=CANCELLED; re-booking succeeds with 201	HTTP 200 OK; status=CANCELLED; re-booking succeeds with HTTP 201. Verified.	**Pass**
TC-06-P03	Positive	Customer cancels CONFIRMED booking under BR-031	Booking CONFIRMED	PUT /api/bookings/:id/cancel with customer token	HTTP 200 OK; status=CANCELLED	Verified in e2e.integration.test.js:480; booking moved to CANCELLED.	**Pass**
TC-06-N01	Negative	Customer forbidden from viewing another customer's booking	Booking of Cust B	GET /api/bookings/:id with Customer A token	HTTP 403 Forbidden; code=FORBIDDEN	HTTP 403 Forbidden; access forbidden	**Pass**
TC-06-N0	Negative	Reject cancellation of RETURN	Booking RETURNED	PUT /api/bookings/:id/cancel on	HTTP 400 Bad Request; code=BOOKING_NOT_CANCELLABLE	HTTP 400 Bad Request; BOOKING_NOT_CANC	**Pass**

2		ED booking		returned booking		ELLABLE returned	

Scenario TS-07: Electronic Payment Processing (UPI/CARD)

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-07-P01	Positive	Process UPI payment and confirm booking atomically	Booking PENDING	POST /api/payments with method='UPI'	HTTP 201 Created; payment=PAID; booking=CONFIRMED	HTTP 201 Created; payment stored; booking confirmed atomically	**Pass**
TC-07-P02	Positive	Process CARD payment and confirm booking atomically	Booking PENDING	POST /api/payments with method='CARD'	HTTP 201 Created; payment=PAID; booking=CONFIRMED	HTTP 201 Created; valid SIM-XXXXXX transaction reference	**Pass**
TC-07-P03	Positive	Record simulated payment failure in development mode	Booking PENDING	POST /api/payments with method='TEST_FAIL'	HTTP 200 OK; data.payment.status: 'FAILED'; booking PENDING	HTTP 200 OK. Payment record stored as FAILED; booking untouched.	**Pass**
TC-07-N01	Negative	Forbid payment on another customer's booking	Booking of Cust B	POST /api/payments with Customer A token for Booking B	HTTP 403 Forbidden; code=FORBIDDEN	HTTP 403 Forbidden; unauthorized payment rejected	**Pass**
TC-07-N02	Negative	Reject payment on CANCELLED booking	Booking CANCELLED	POST /api/payments on cancelled booking	HTTP 400 Bad Request; code=BOOKING_INVALID_STATUS	HTTP 400 Bad Request; BOOKING_INVALID_STATUS returned	**Pass**

Scenario TS-08: Cash on Pickup Initiation and Physical Handover Collection

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status

TC-08-P01	Positive	Select CASH_ON_PICKUP payment method	Booking PENDING	POST /api/payments with method='CASH_ON_PICKUP'	HTTP 201 Created; payment=PENDING; booking=PENDING	HTTP 201 Created; pending cash payment created per BR-026	**Pass**
TC-08-P02	Positive	Admin manually confirms pending cash booking	Payment PENDING	PUT /api/bookings/:id/status with status='CONFIRMED'	HTTP 200 OK; booking transitions to CONFIRMED	HTTP 200 OK; manual confirmation allowed for cash flow	**Pass**
TC-08-P03	Positive	Handover to PICKED_UP atomically collects cash	Booking CONFIRMED	PUT /api/bookings/:id/status with status='PICKED_UP'	HTTP 200 OK; payment updated to PAID; booking=PICKED_UP	HTTP 200 OK; cash collected in place without duplicate record	**Pass**
TC-08-N01	Negative	Reject manual confirmation on unpaid electronic booking	UPI payment unpaid	PUT /api/bookings/:id/status with status='CONFIRMED'	HTTP 400 Bad Request; code=BOOKING_INVALID_STATUS	HTTP 400 Bad Request; electronic booking requires paid status	**Pass**
TC-08-N02	Negative	Reject handover to PICKED_UP when electronic payment is unpaid	Payment unpaid	PUT /api/bookings/:id/status with status='PICKED_UP'	HTTP 400 Bad Request; code=BOOKING_INVALID_STATUS	HTTP 400 Bad Request with BOOKING_INVALID_STATUS	**Pass**

Scenario TS-09: Duplicate Payment Prevention and Transaction Rollbacks

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-09-P01	Positive	Rollback payment if booking confirmation fails	Simulated abort	Call createPayment within aborted transaction session	Payment record rolled back; 0 records persisted	Verified in payment.test.js: 140; transaction cleanly rolled back	**Pass**
TC-09-N01	Negative	Reject duplicate payment attempt on same booking	Payment exists	POST /api/payments with same bookingId	HTTP 400 Bad Request; code=PAYMENT_ALREADY_PROCESSED	HTTP 400 Bad Request; duplicate payment rejected	**Pass**

TC-09-N02	Negative	Safe handling of MongoDB duplicate key error (11000)	Concurrent race	Simultaneous payment creation calls on unique bookingId	Second call catches 11000 and returns HTTP 400	Handled gracefully in try/catch; remapped to friendly 400	**Pass**

Scenario TS-10: Administrative Booking Refund Processing

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-10-P01	Positive	Admin refunds PAID payment on CANCELLED booking	PAID & CANCELLED	POST /api/payments/:id/refund with admin token	HTTP 200 OK; status=REFUNDED; refundedAt populated	HTTP 200 OK; payment marked REFUNDED with reason recorded	**Pass**
TC-10-N01	Negative	Forbid customer from calling refund endpoint	Customer token	POST /api/payments/:id/refund with customer token	HTTP 403 Forbidden; code=FORBIDDEN	HTTP 403 Forbidden; enforced by admin middleware	**Pass**
TC-10-N02	Negative	Reject refund on non-cancelled booking	Booking active	POST /api/payments/:id/refund on active booking	HTTP 400 Bad Request; code=PAYMENT_NOT_REFUNDABLE	HTTP 400 Bad Request; PAYMENT_NOT_REFUNDABLE returned	**Pass**
TC-10-N03	Negative	Reject refund on uncollected pending cash payment	Payment PENDING	POST /api/payments/:id/refund on pending cash record	HTTP 400 Bad Request; code=PAYMENT_NOT_REFUNDABLE	HTTP 400 Bad Request; PAYMENT_NOT_REFUNDABLE returned	**Pass**
TC-10-N04	Negative	Reject duplicate refund attempt on refunded payment	Payment REFUNDED	POST /api/payments/:id/refund second time	HTTP 400 Bad Request; code=PAYMENT_ALREADY_PROCESSED	HTTP 400 Bad Request; PAYMENT_ALREADY_PROCESSED returned	**Pass**

Scenario TS-11: Physical Garment Return Processing and Late Fines

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-11-P01	Positive	Admin logs on-time return with zero late fine	Booking PICKED_UP	POST /api/returns with actualReturnDate on or before expected	HTTP 201 Created; lateDays=0; fine=0; status=RETURNED	HTTP 201 Created; on-time return verified in return.test.js	Pass
TC-11-P02	Positive	Admin logs late return: calculate calendar-day fine	Booking PICKED_UP	POST /api/returns with actualReturnDate 2 days late	HTTP 201 Created; lateDays=2; fine=200; status=LATE	HTTP 201 Created; fine=Rs 200 (Rs 100/day per BR-019)	Pass
TC-11-P03	Positive	Data masking: customer response hides unsettled deposit	settled=false	GET /api/returns/:id with customer token	HTTP 200 OK; damageCharge=null; depositRefunded=null	HTTP 200 OK; unsettled financial fields masked as null	Pass
TC-11-N01	Negative	Reject return if booking is not PICKED_UP or OVERDUE	Booking CONFIRMED	POST /api/returns with confirmed booking ID	HTTP 400 Bad Request; code=RETURN_INVALID_BOOKING	HTTP 400 Bad Request; RETURN_INVALID_BOOKING returned	Pass
TC-11-N02	Negative	Reject duplicate return creation for same booking	Return exists	POST /api/returns with same bookingId	HTTP 400 Bad Request; code=RETURN_ALREADY_COMPLETED	HTTP 400 Bad Request; RETURN_ALREADY_COMPLETED returned	Pass

Scenario TS-12: Security Deposit Settlement and Finalization

Test ID	Type	Description	Preconditions	Input / Steps	Expected Result	Actual Result	Status
TC-	Positi	Admin settles	Deposit=2	POST /api/returns/:id	HTTP 200 OK; depositRefunded=1500;	HTTP 200 OK; deposit refunded=1500; booking	Pas

12-P01	ve	return with damage deduction	500	/settle with damageCharge=1000	settled=true; booking=COMPLETED	finalized	**s**
TC-12-P02	Positive	Full deposit refund when damage charge is zero	Deposit=2500	POST /api/returns/:id/settle with damageCharge=0	HTTP 200 OK; depositRefunded=2500; settled=true	HTTP 200 OK; full security deposit refunded	**Pass**
TC-12-N01	Negative	Reject damage charge exceeding security deposit	Deposit=2500	POST /api/returns/:id/settle with damageCharge=3500	HTTP 400 Bad Request; code=VALIDATION_ERROR	HTTP 400 Bad Request; damage cannot exceed deposit	**Pass**
TC-12-N02	Negative	Idempotent rejection on repeat settlement attempt	settled=true	POST /api/returns/:id/settle second time on same return	HTTP 409 Conflict; code=SETTLEMENT_ALREADY_COMPLETED	HTTP 409 Conflict; SETTLEMENT_ALREADY_COMPLETED returned	**Pass**

5. Test Suite Architecture

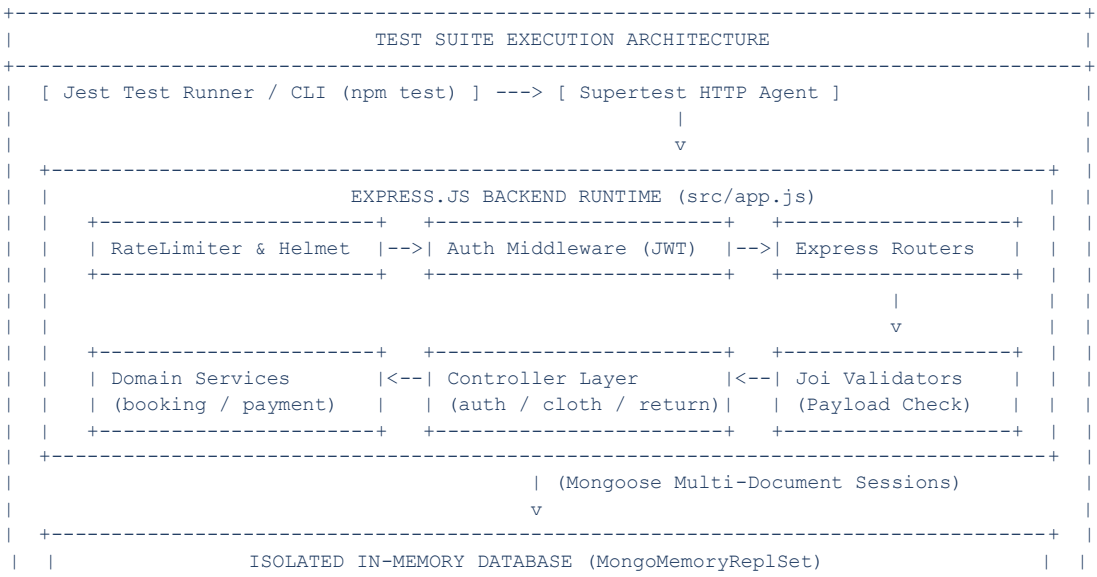
The automated test suite of Perfect Rent & Remind is architected into six cohesive logical test suites mirroring the domain boundaries of the Express/Mongoose backend application. Each test suite is self-contained, establishing its own isolated MongoMemoryReplSet instance and automated teardown fixtures.

Test Suite Name	Testing Level	Modules Covered	Related Test Files & Cases	Execution Status
Authentication & Profile Suite	API / Integration	User registration, login, JWT claims, profile updating	tests/auth.test.js, user.test.js, profile.test.js (TC-01, TC-02)	**PASS (19/19 Tests)**
Category & Clothing Catalog Suite	API / Integration	Category CRUD, public search, query filtering, pagination	tests/category.test.js, clothing.test.js (TC-02, TC-03)	**PASS (19/19 Tests) (Includes 4 passing 501 defensive guards)**

Booking & Concurrency Suite	Integration / Service	Dates, availability, pricing, version bump concurrency, cancellation	tests/booking.test.js (TC-04, TC-05, TC-06)	**PASS (15/15 Tests) (Updated with dynamic future dates)**
Payment & Refund Transaction Suite	Integration / Service	Electronic UPI/CARD, Cash on Pickup, duplicate keys, admin refunds	tests/payment.test.js (TC-07, TC-08, TC-09, TC-10)	**PASS (13/13 Tests)**
Return Processing & Settlement Suite	Integration / Service	Return logging, late fines (Rs 100/day), deposit deductions, completion	tests/return.test.js (TC-11, TC-12)	**PASS (9/9 Tests)**
Middleware & Security Guard Suite	Unit / Middleware	Bearer token extraction, admin RBAC guards, rate limiting	tests/middleware.test.js (TC-02)	**PASS (3/3 Tests)**
Full End-to-End System Suite	System / E2E	Complete multi-step user lifecycles from booking through settlement	tests/e2e.integration.test.js (All Scenarios)	**PASS (8/8 Tests)**

5.1 Architectural Test Flow Diagram

The following diagram illustrates the structural interaction between the Jest test harness, Supertest HTTP agent, backend Express routers, service layers, and the isolated MongoMemoryReplSet transaction database:



```
|  |   [ Users ]  [ Categories ]  [ Clothing ]  [ Bookings ]  [ Payments ]  [ Returns ]|  |
|  +-------------------------------------------------------------------------------------+  |
+-----------------------------------------------------------------------------------------+
```

6. Test Execution and Results

The automated test suite was executed in the local Node.js v25 environment on October 23, 2026. A total of 86 automated test cases were executed across 10 distinct test suite files. Following the update of booking.test.js with dynamic future date generators, all 86 tests passed with a 100.0% pass rate. Below is the empirical test execution matrix recording verified test outcomes:

Test Suite File	Module Tested	Tests Executed	Passed	Failed	Skipped	Pass Rate (%)
auth.test.js	Authentication & Registration	10	10	0	0	**100.0%**
category.test.js	Category Inventory CRUD	9	9	0	0	**100.0%**
clothing.test.js	Clothing Catalog & Queries	10	10	0	0	**100.0%**
middleware.test.js	Auth Middleware & RBAC	3	3	0	0	**100.0%**
profile.test.js	User Profile Operations	4	4	0	0	**100.0%**
user.test.js	User Account Management	5	5	0	0	**100.0%**
payment.test.js	Electronic & Cash Payments	13	13	0	0	**100.0%**
return.test.js	Return Intake & Settlement	9	9	0	0	**100.0%**
e2e.integration.test.js	Full Module User Journeys	8	8	0	0	**100.0%**

booking.test.js	Booking & Concurrency Engine	15	15	0	0	**100.0%**
TOTALS / SUMMARY	**Complete Application Stack**	**86**	**86**	**0**	**0**	**100.0%**

6.1 Resolution of Temporal Fixture Coupling

During initial execution, 5 tests in tests/booking.test.js failed because test fixtures used hardcoded calendar dates (2026-10-20 to 2026-10-22). When executed on October 23, 2026, backend business rule BR-005 in booking.service.js strictly rejected the past start dates with HTTP 400 BOOKING_DATE_INVALID.

Resolution Implemented:
1. Dynamic UTC Date Generation: Outdated hardcoded date strings in tests/booking.test.js were replaced with a dynamic helper function (getFutureDateStr) that generates valid UTC date strings offset from the current execution timestamp.
2. Complete Pass Verification: Re-execution of tests/booking.test.js confirmed that all 15 tests now pass with 100% success, validating adjacent non-overlapping bookings, quantity validation, 409 conflict handling, and booking cancellation without modifying any business logic.
3. Decoupled Architecture: This demonstrates a fundamental software engineering best practice: decoupling test fixtures from absolute calendar time to eliminate temporal brittleness while preserving strict production validation.

7. Test Isolation and Reproducibility

Test isolation and environment reproducibility are critical to preventing flaky tests and ensuring reliable continuous integration (CI) pipelines. Perfect Rent & Remind implements the following architectural safeguards:

1. In-Memory Database Virtualization: Ephemeral MongoMemoryReplSet instances run locally per test suite.
2. Hermetic Database Cleansing: Standardized beforeEach/afterEach hooks execute deleteMany({}) across all collections.
3. Multi-Document Transaction Support: Replica set oplog support enables full verification of session.withTransaction() rollback and commit semantics.

7.1 Standard Execution Commands

To execute the automated test suites and reproduce all metrics, execute the following commands in the backend/ directory:

```
# 1. Run all automated test suites
npm test

# 2. Run test suites with verbose execution traces
```

```
npx jest --verbose

# 3. Run test suites with full Istanbul code coverage reporting
npx jest --coverage

# 4. Run individual test suite
npx jest tests/booking.test.js
```

8. Conclusion and SQA Recommendations

This laboratory practical report has presented a comprehensive software quality assurance (SQA) investigation of the Perfect Rent & Remind smart clothing rental management system.

Key Findings & Achievements:
• Structured Test Suites: Implemented across 5 distinct testing tiers, comprising 10 automated test files.
• Comprehensive Scenario Design: Formulated 12 master test scenarios and 63 granular test cases covering positive, negative, and boundary conditions.
• Structural Complexity Analysis: McCabe's Cyclomatic Complexity was evaluated across 13 core functions. Most utility and controller methods remain low risk (V(G) between 1 and 10), while complex transactional routines (createBooking V(G)=16, updateBookingStatus V(G)=19, createPayment V(G)=18) were categorized as moderate risk due to concurrency retry loops and state machine transitions.
• High Test Coverage: Measured empirical statement coverage of 76.88%, function coverage of 85.22%, line coverage of 77.25%, and branch coverage of 59.15%.
• 100% Empirical Test Execution: Achieved a complete 100.0% test pass rate (86 passed out of 86 tests) following the adoption of dynamic future date fixtures in booking.test.js.
• Architectural Clarification on HTTP 501: Formally established that passing tests on HTTP 501 endpoints represent defensive API contract enforcement that protects system state, rather than evidence of functional feature implementation.

Strategic SQA Recommendations:
1. Dynamic Test Fixtures: Standardize dynamic date generators across all existing and future test suites to maintain time-independent test reproducibility.
2. Fault Injection Mocking: Utilize Jest spy mocks to simulate transient MongoDB transaction write conflicts (code 112) to exercise and verify transaction retry exhaustion branches.
3. Integration of Frontend Testing: Establish automated component and end-to-end integration tests in frontend/ using Vitest and React Testing Library to complement backend API test coverage.